



UNIVERSIDAD
NACIONAL
DE LA PLATA

Minería de Datos y Aprendizaje Automático

Clase 4
Métricas de evaluación - Ensembles

2026

Temario

1

Métricas de evaluación

2

Ensembles

3

Bagging

4

Boosting

5

Stacking

01

Métricas de evaluación



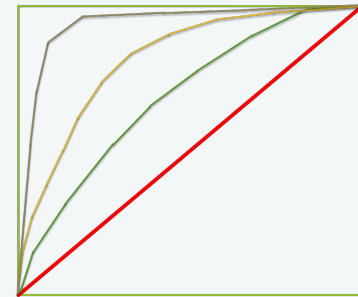
Métricas de evaluación

Métricas en problemas de clasificación:

- Matriz de confusión
- Exactitud (Accuracy)
- Precisión (Precision)
- Exhaustividad (Recall)
- F-score
- Especificidad (Specificity)
- Tasa de falsos negativos (False negative rate)
- Tasa de falsos positivos (False positive rate)
- Curvas ROC (Receiver Operating Characteristic)
- Curvas Precision-Recall
- Área bajo la curva
- *Algunas más...*

		Clase predicha				
		Clase A	Clase B	Clase C	Clase D	Clase E
Clase real	Clase A	TN	FP	TN	TN	TN
	Clase B (positivo)	FN	TP	FN	FN	FN
	Clase C	TN	FP	TN	TN	TN
	Clase D	TN	FP	TN	TN	TN
	Clase E	TN	FP	TN	TN	TN

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP Verdaderos positivos	FN Falsos negativos
	Negativo (N)	FP Falsos positivos	TN Verdaderos negativos



Matriz de confusión

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP Verdaderos positivos	FN Falsos negativos
	Negativo (N)	FP Falsos positivos	TN Verdaderos negativos

En problemas de clasificación binario, la matriz de confusión nos muestra para cada clase, los aciertos y fallos de las predicciones.

Por lo general, en estos problemas se trabaja con una clase positiva y otra negativa.

Ej: ausencia o presencia de una enfermedad.

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	154	0
	Negativo	0	168

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	109	45
	Negativo	16	152

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	79	75
	Negativo	86	82

Exactitud (Accuracy)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{(TP+TN)}{(TP+FN+FP+TN)}$$

¿Qué tanto acierta el modelo?

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

Accuracy 97,52%
¿Nos sirve este modelo?

Precisión (Precision)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{TP}{(TP+FP)}$$

Cuando predice la clase positivo ¿Qué tan seguro está?

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

Precision 48,78%

Exhaustividad (Recall)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{TP}{(TP+FN)}$$

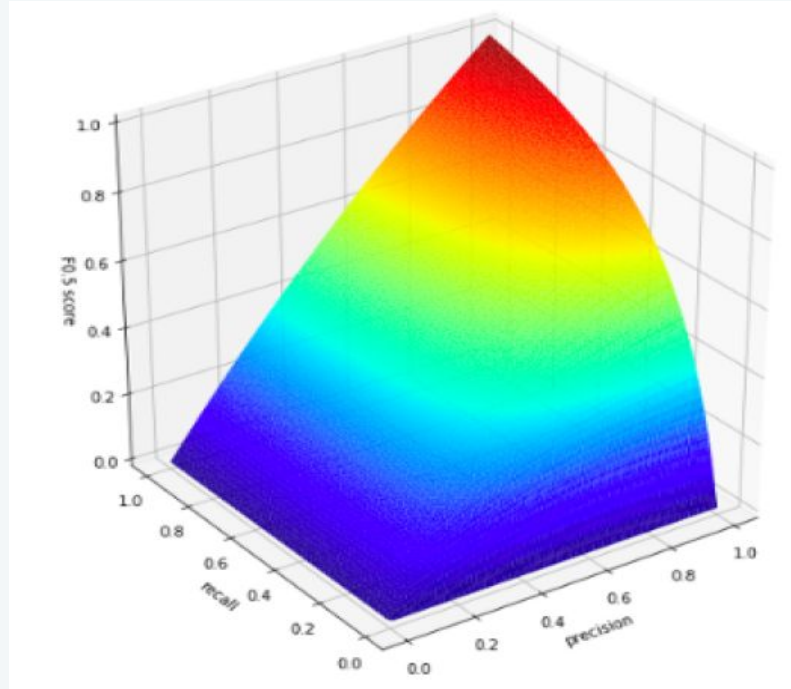
También llamada
sensibilidad (sensitivity)

¿Cuánto acierta de la
clase positivo?

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

Recall 80,00%

Medida F (F-score)



Media ponderada entre
precisión y exhaustividad

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

F-Score 60,61%

$$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Especificidad (Specificity)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{TN}{(FP+TN)}$$

¿Cuánto acierta de la clase negativo?

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

Specificity 97,94%

Tasa de falsos negativos (False negative rate)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{FN}{(TP+FN)}$$

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

FNR 20,00%

¿En qué proporción dice el modelo que una muestra es negativa, cuando no lo es?

Tasa de falsos positivos (False positive rate)

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo (P)	TP	FN
	Negativo (N)	FP	TN

$$\frac{FP}{(FP+TN)}$$

		Clase predicha	
		Positivo	Negativo
Clase real	Positivo	20	5
	Negativo	21	1002

FPR 02,05%

¿En qué proporción dice el modelo que una muestra es positiva, cuando no lo es?

Exactitud (Accuracy)

Problemas multiclase

		Clase predicha				
		Clase A	Clase B	Clase C	Clase D	Clase E
Clase real	Clase A	AA				
	Clase B		BB			
	Clase C			CC		
	Clase D				DD	
	Clase E					EE

$$\frac{(AA+BB+CC+DD+EE)}{\textit{total de muestras}}$$

Métricas restantes

Problemas multiclase

		Clase predicha				
		Clase A	Clase B	Clase C	Clase D	Clase E
Clase real	Clase A	TN	FP	TN	TN	TN
	Clase B	FN	TP	FN	FN	FN
	Clase C	TN	FP	TN	TN	TN
	Clase D	TN	FP	TN	TN	TN
	Clase E	TN	FP	TN	TN	TN



		Clase predicha	
		Clase B	Clase ACDE
Clase real	Clase B (positivo)	TP	Σ FN
	Clase ACDE (negativo)	Σ FP	Σ TN

Cálculo de métricas

```
from sklearn import metrics
```

```
cm = metrics.confusion_matrix(T_test, T_pred)
```

```
print(cm)
```

```
metrics.ConfusionMatrixDisplay(confusion_matrix=cm).plot();
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion\_matrix.html
```

```
print(metrics.precision_score(T_test, T_pred))
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision\_score.html
```

```
print(metrics.recall_score(T_test, T_pred))
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall\_score.html
```

```
print(metrics.f1_score(T_test, T_pred))
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1\_score.html
```

```
p,r,f,s = metrics.precision_recall_fscore_support(T_test, T_pred)
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision\_recall\_fscore\_support.html
```

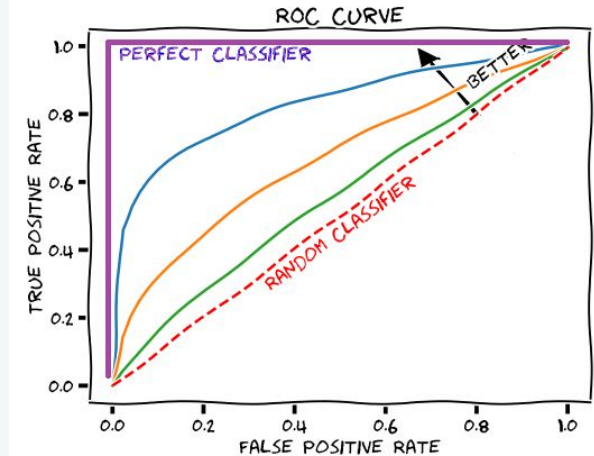
Curvas ROC

Una curva ROC (Receiver Operating Characteristic (Característica Operativa del Receptor)) es una representación gráfica del rendimiento de un modelo de clasificación binaria para todos los posibles umbrales de clasificación.

Permite visualizar la sensibilidad versus la especificidad de un modelo.

Sensibilidad $\rightarrow$ TPR = $TP / (TP + FN)$

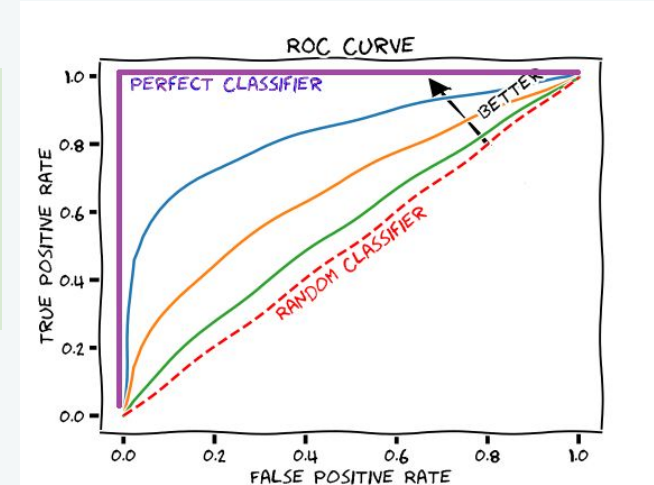
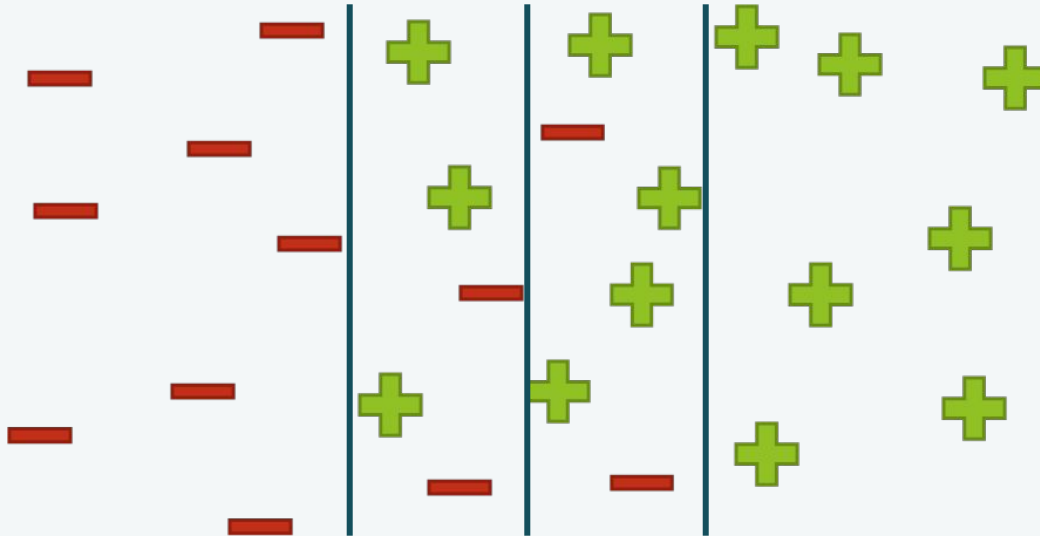
1 - Especificidad $\rightarrow$ FPR = $FP / (FP + TN)$ =
 $= 1 - TN / (FP + TN)$



TP	FN
FP	TN

Curvas ROC

Muchos algoritmos trabajan con un umbral de decisión. Según donde se encuentre ese umbral, el error puede mejorar la TPR, empeorando la FPR; o viceversa.



$$\begin{aligned} \text{TPR} &= 11 / 14 = 0.78 \\ \text{FPR} &= 1 - 10 / 12 = 0.17 \end{aligned}$$

$$\begin{aligned} \text{TPR} &= 7 / 14 = 0.5 \\ \text{FPR} &= 1 - 12 / 12 = 0 \end{aligned}$$

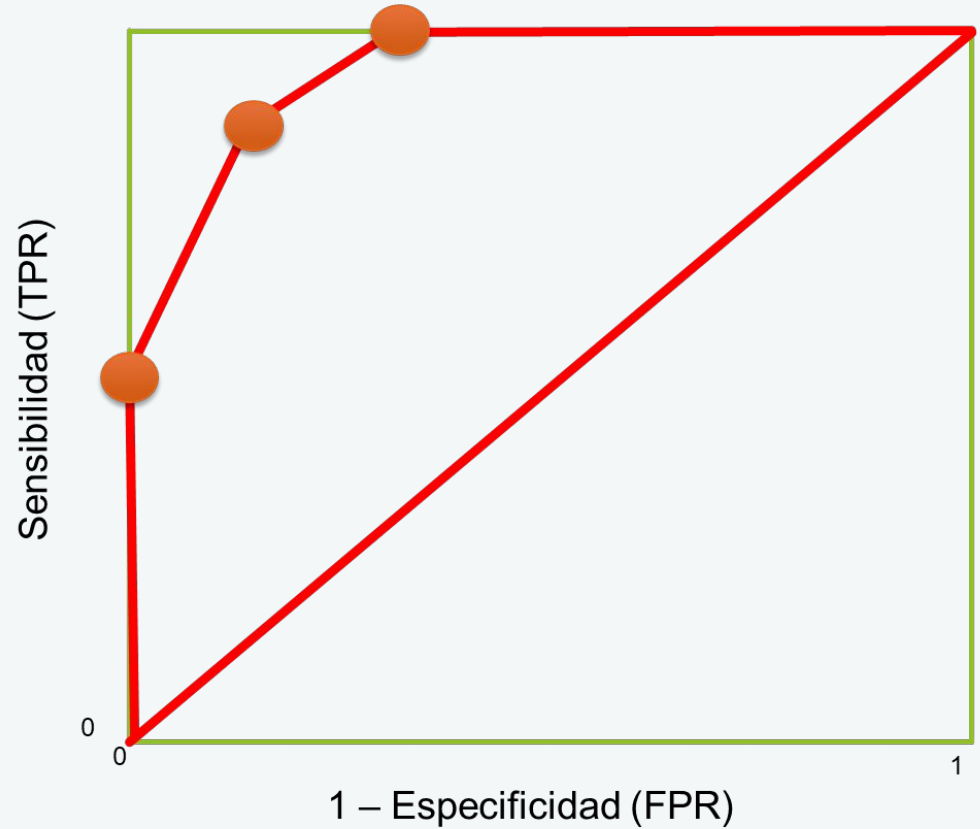
$$\begin{aligned} \text{TPR} &= 14 / 14 = 1 \\ \text{FPR} &= 1 - 8 / 12 = 0.33 \end{aligned}$$

Curvas ROC

$$\text{TPR} = 11 / 14 = 0.78$$
$$\text{FPR} = 1 - 10 / 12 = 0.17$$

$$\text{TPR} = 7 / 14 = 0.5$$
$$\text{FPR} = 1 - 12 / 12 = 0$$

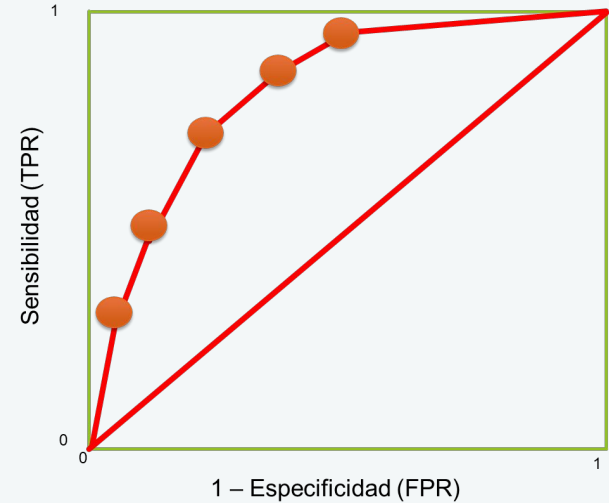
$$\text{TPR} = 14 / 14 = 1$$
$$\text{FPR} = 1 - 8 / 12 = 0.33$$



Curvas ROC

Si el algoritmo da una respuesta probabilística, también se puede hacer el mismo ejercicio, pero modificando el umbral de redondeo.

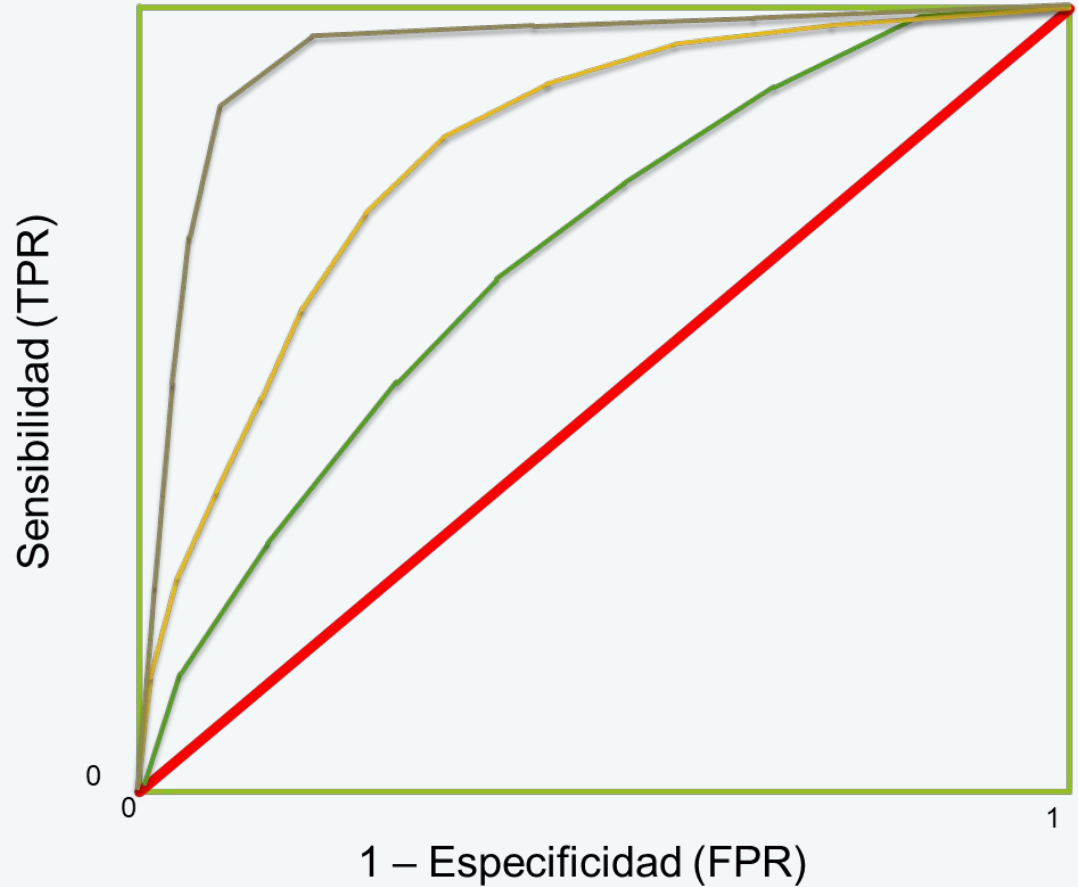
Target	Prob. clase 1	Umbral 0.2	Umbral 0.4	Umbral 0.5	Umbral 0.7	Umbral 0.85
1	0.8	1	1	1	1	0
0	0.3	1	0	0	0	0
1	0.6	1	1	1	0	0
0	0.2	1	0	0	0	0
1	0.7	1	1	1	1	0
1	0.9	1	1	1	1	1
0	0.4	1	1	0	0	0



Curvas ROC

Cuanto más cercana esté del (0,1) mejor será el modelo.

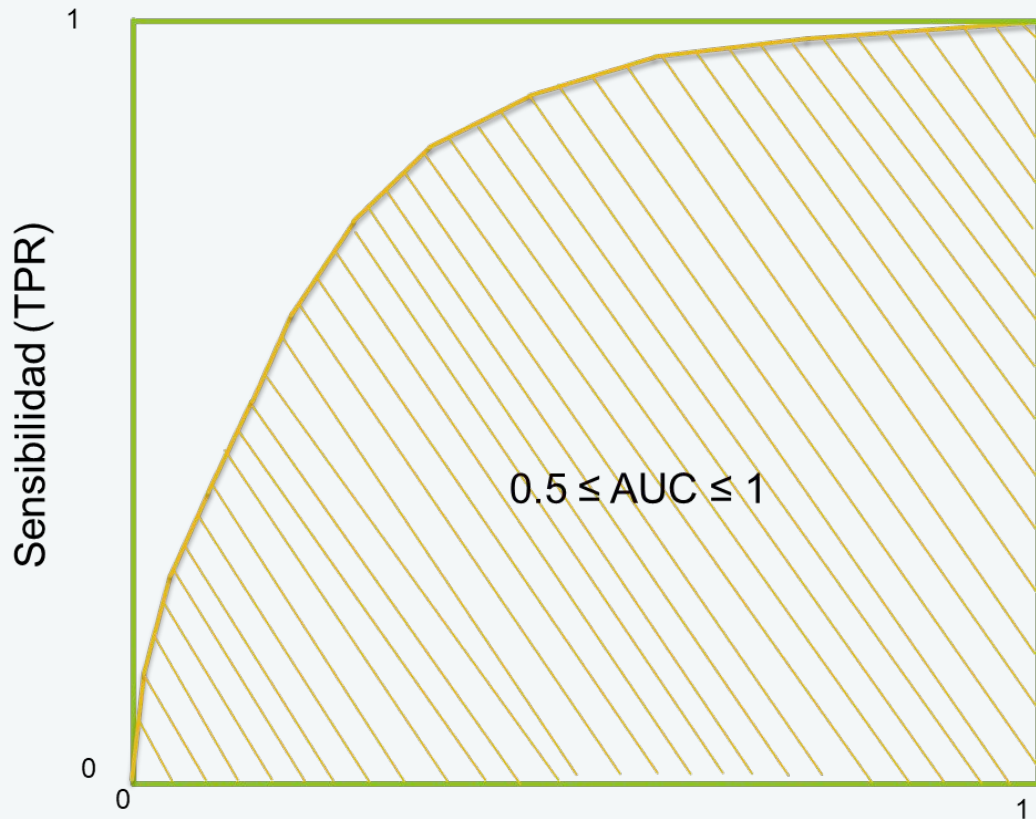
La línea roja marca la eficacia del azar.



AUC ROC

El área bajo la curva ROC (AUC ROC) nos da un indicador del modelo.

Es utilizado para comparar poder pronóstico entre diferentes modelos.



Curvas ROC

```
from sklearn.metrics import roc_curve, auc
```

```
dt = DecisionTreeClassifier(max_depth = 4)  
model = dt.fit(X_train, T_train)
```

```
# Para las curvas ROC necesitamos las probabilidades devueltas para cada muestra  
probs = model.predict_proba(X_test)
```

```
# Calcula la curva ROC  
fpr, tpr, thresholds = roc_curve(T_test, probs)
```

```
# Calcula el área bajo la curva  
roc_auc = auc(fpr, tpr)
```

```
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc\_curve.html
```

Curvas ROC

```
import matplotlib.pyplot as plt

# Para graficar la curva ROC
probs = model.predict_proba(X_test)

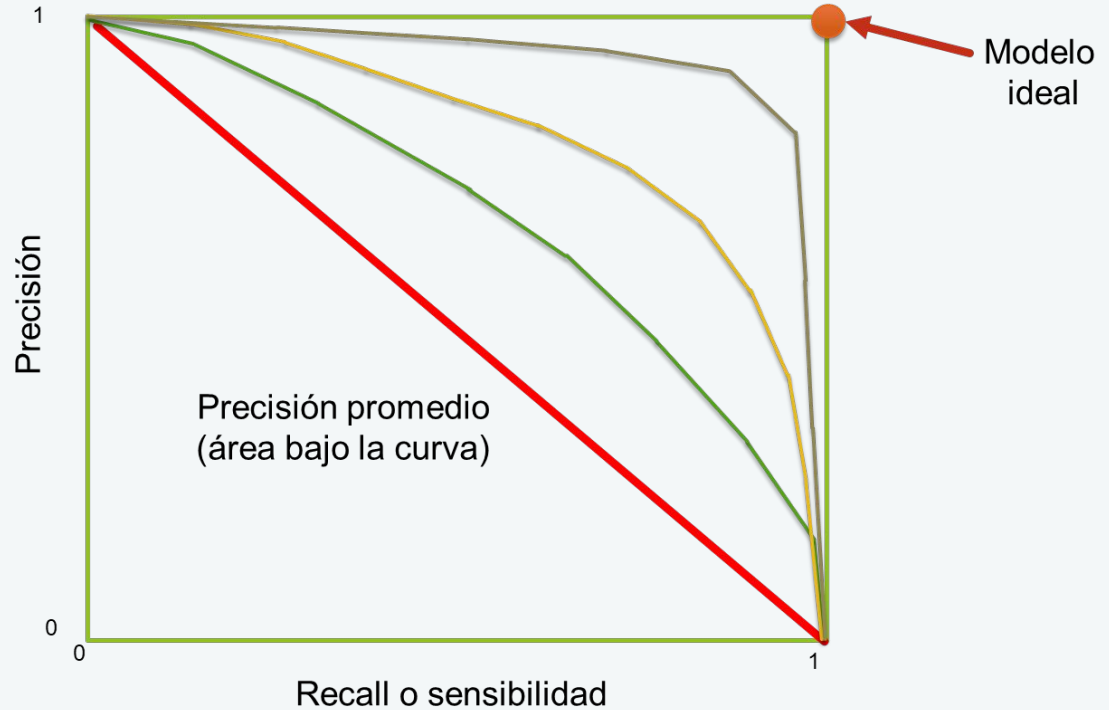
plt.figure()
plt.plot(fpr, tpr, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], 'k--', label='No Skill')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Breast Cancer Classification')
plt.legend()
plt.show()
```

Curvas Precision-Recall

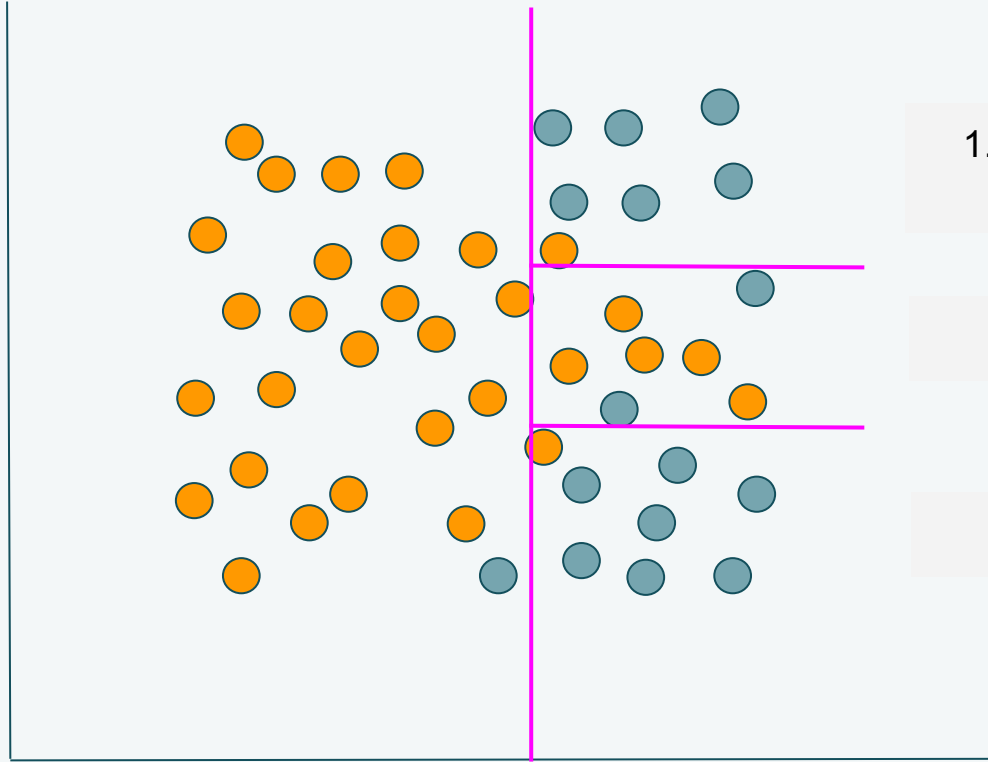
Otra curva que suele utilizarse es la curva PR (Precision-Recall).

Se construye de la misma manera que la curva ROC, pero utilizando las métricas precision y recall.

Dadas las características de las métricas, el modelo ideal está en (1,1).



Evaluando al modelo

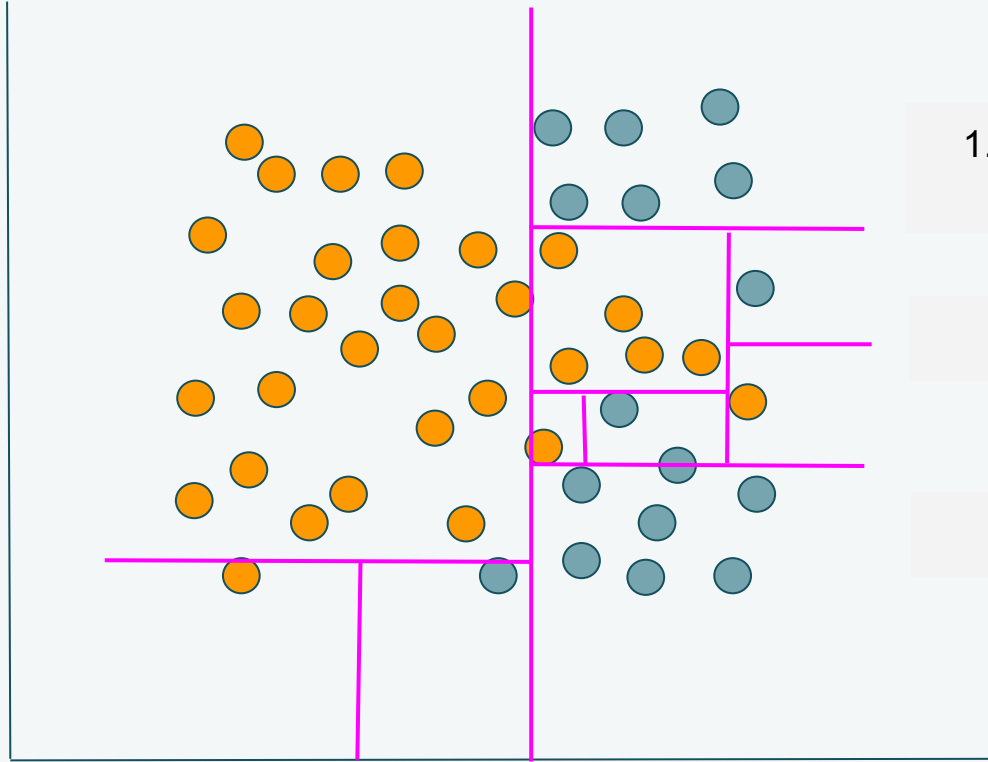


1. Separamos en train-test
(`random_state = 13`)

2. Entrenamos

3. Testeamos

Evaluando al modelo



1. Separamos en train-test
(`random_state = 27`)

2. Entrenamos

3. Testeamos

Rigor estadístico

Bootstrapping

1. Generar **B** nuevos conjuntos de prueba (normalmente 1000) haciendo muestreo con reemplazo del conjunto original.
Cada nuevo conjunto tendrá el mismo tamaño **N** pero algunos datos se repetirán y otros no aparecerán.
2. Evaluar la/s métrica/s a utilizar en cada uno de los **B** ensayos, obteniendo así 1000 valores diferentes.
3. Calcular el promedio de cada métrica y su intervalo de confianza.

En estadística, la ley de los grandes números menciona un mínimo de 30 ensayos distintos, para asegurar que el promedio empiece a “estabilizarse” en su valor final. Aunque siempre dependerá de las características del dataset a analizar.

Intervalo de confianza del 95%

$$X \pm 1.96 \frac{\sigma}{\sqrt{n}}$$

Ahora... ya alcancé un promedio e IC de las métricas aceptable
¿Con cuál de los 1000 modelos me quedo?



Actividades

1. Utilice la base de datos “glass”, separando en 90-10.

Calcule e imprima la matriz de confusión.

Calcule e imprima accuracy, precision, recall, f-score y AUC-ROC.

Repita los cálculos 10 veces, cambiando el random_state en el split. Calcule el promedio e IC_95% de cada métrica.

02

Ensembles



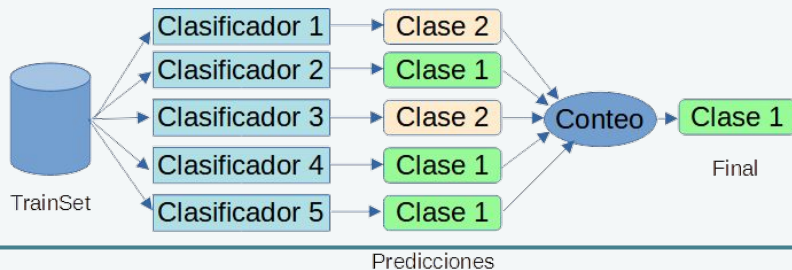
Ensembles

Los ensambles (ensembles) consiste en un conjunto de modelos clasificadores individuales (conocidos como estimadores base), los cuales todos juntos aportan al mecanismo de predicción.

Los ensambles conforman una solución que supera los desafíos técnicos que implica la creación de un único estimador.

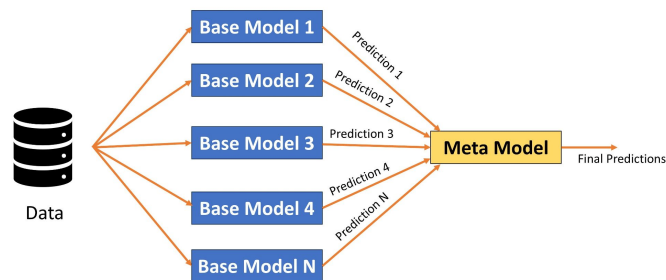
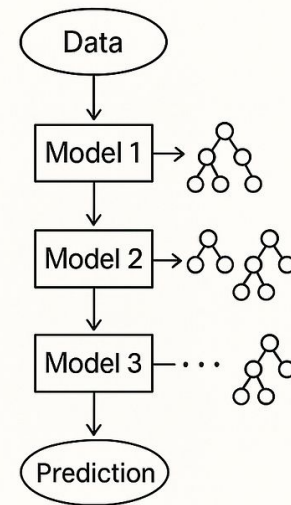
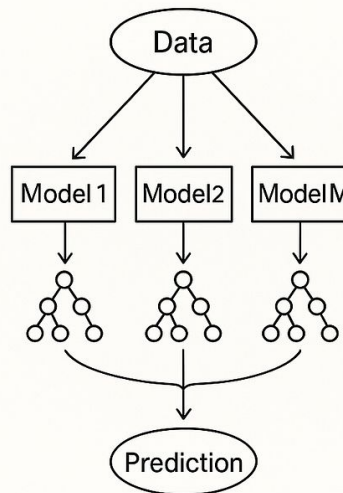
Ventajas:

- Reducen el error en la predicción.
- Mantienen una buena generalización.



Ensembles

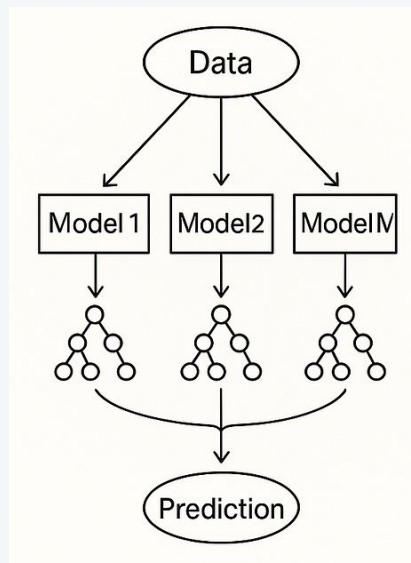
- Bagging
 - Random forest
- Boosting
 - AdaBoost
 - Gradient boosting
 - XGBoost
- Stacking



03

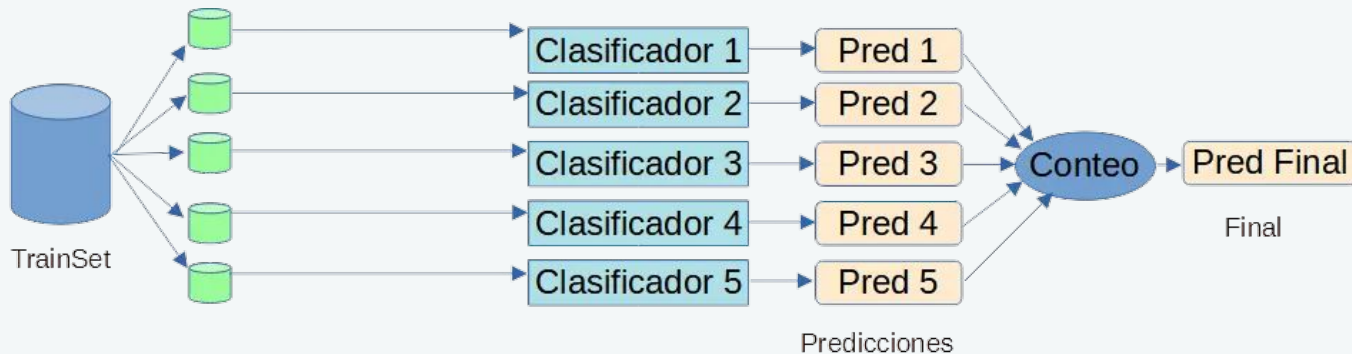
Bagging

Random Forest



Bagging

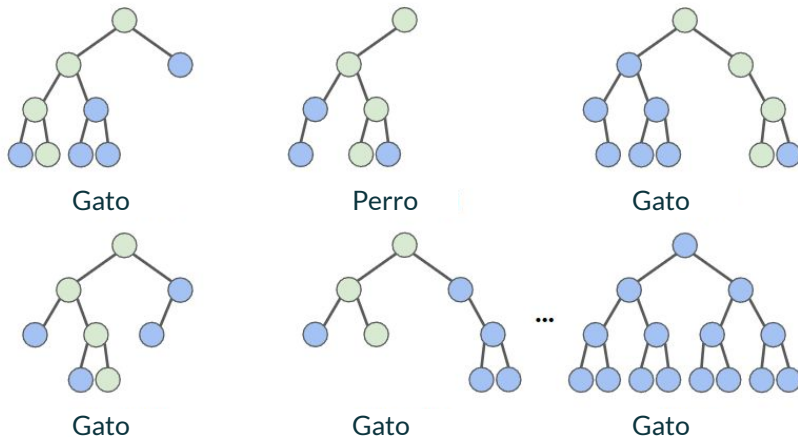
- Se entrenan N estimadores diferentes.
 - Bootstrapping: se seleccionan al azar las filas del dataset (con reemplazo).
 - Pasting: se seleccionan al azar las filas del dataset (sin reemplazo).
 - Random subspace: se seleccionan al azar las columnas (atributos) del dataset.
- La respuesta más votada es la respuesta del ensemble.



Random Forest

- Es un ensamble de N árboles independientes.
- Cada árbol se construye con un número distinto de muestras y de atributos.
- La predicción se realiza con el mecanismo de votación.

Peso: 6000 gr.
Salto: 180 cm.
Dientes: 28
Comida: 350 gr.



Gato ($\frac{5}{6}$)

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators = 1000)
model = rf.fit(X_train, T_train)

T_pred = model.predict(X_test)

e = rf.estimators_[0] # permite obtener los árboles (zero-based),
                      # por ejemplo para visualizarlos.

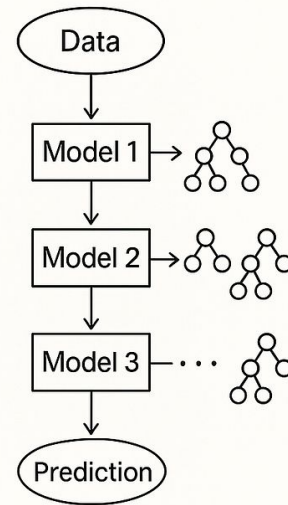
# devuelve la proporción de votos
print(model.predict_proba(X_test))

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
```

```
[0.13 0.87]
[1.   0.  ]
[0.88 0.12]
[0.51 0.49]
[0.1  0.9  ]
[1.   0.  ]
[0.96 0.04]
[0.89 0.11]
[0.16 0.84]
[0.18 0.82]
[0.1  0.9  ]
[0.14 0.86]
[0.05 0.95]
[0.99 0.01]
[0.42 0.58]
[0.46 0.54]
[0.04 0.96]
[0.91 0.09]
[0.98 0.02]
[0.77 0.23]
[0.61 0.39]
```

04

Boosting

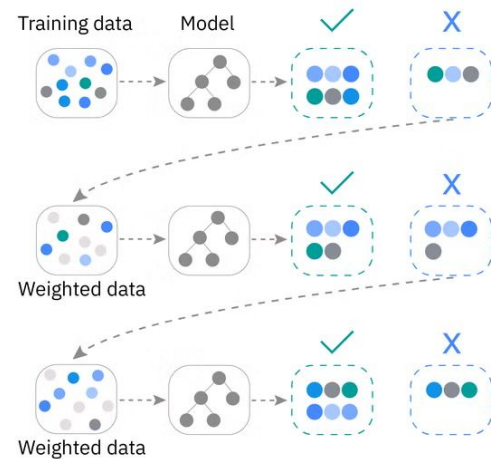


Boosting

El objetivo es usar varios modelos “débiles” para lograr un modelo “fuerte” que tenga un error mínimo.

La forma de armar los modelos es secuencial, uno tras otro, de tal manera que al entrenar un modelo, éste toma en cuenta los errores de su predecesor.

- AdaBoost
- Gradient Boosting
- XGBoost



AdaBoost (*Adaptive boosting*)

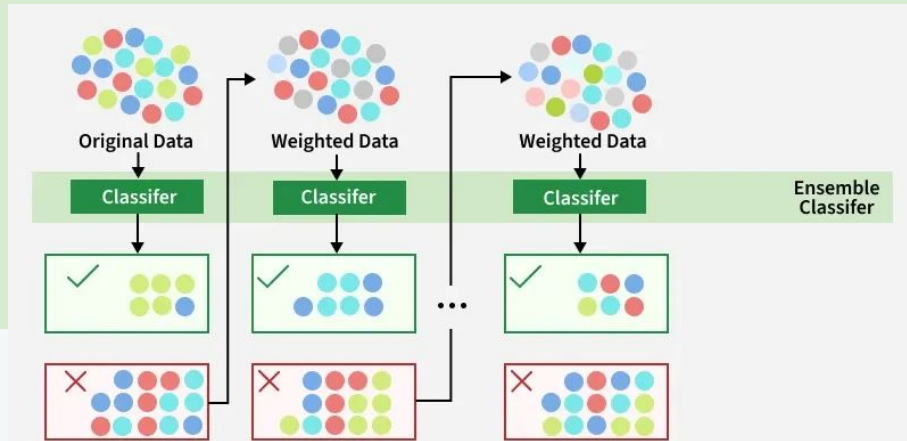
Al comienzo todas las muestras tienen el mismo “peso”.

Se entrena un primer modelo, el cual clasificará correctamente algunas muestras, mientras que otras no.

Las muestras mal clasificadas aumentan su “peso”.

El siguiente modelo a entrenar recibe las mismas muestras, con los pesos modificados. Nuevamente, las mal clasificadas aumentan su “peso”.

Se continúa de la misma manera un número prefijado de pasos...



AdaBoost (*Adaptive boosting*)

Durante el entrenamiento cada modelo obtuvo un “puntaje” de importancia α , según el nivel de precisión alcanzado.

Un modelo con buena performance, obtiene un valor alto de α .

Al momento de realizar la predicción, cada estimador vota de manera independiente, pero la votación se realiza ponderando la respuesta del modelo con su “puntaje” de importancia.

$$voto = \sum \alpha \cdot pred_i(X)$$



AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier

base = DecisionTreeClassifier()

ada = AdaBoostClassifier(estimator=base, n_estimators=500)
model = ada.fit(X_train, T_train)

print(model.estimator_weights_)      # permite obtener el peso de cada
                                     # estimador

print(model.predict_proba(X_test))   # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html
```

Gradient Boosting

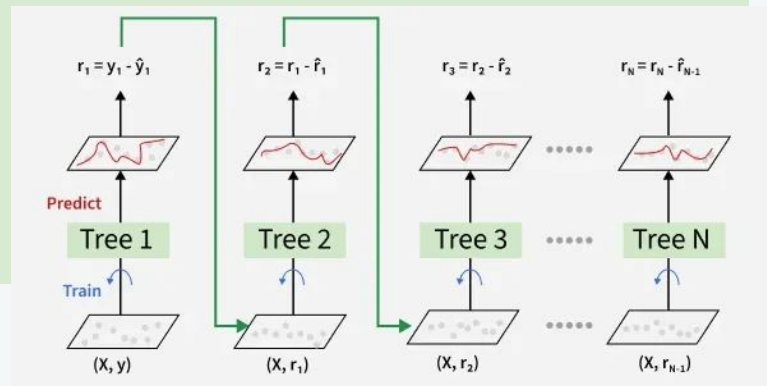
Se entrena un primer modelo (por lo general árboles de decisión), el cual cometerá errores en la clasificación.

Para cada muestra se calcula el “residuo”, que es el error (numérico) que cometió el estimador al hacer la predicción.

Los siguientes modelos “débiles” intentarán predecir esos “residuos”.

Se continúa de la misma manera un número prefijado de pasos.

El resultado es un modelo ensemble “fuerte”.



Gradient Boosting

Se comienza armando un primer clasificador débil, el más simple de todos.

Calcula la probabilidad de que una muestra sea de la clase “1”.

$$Prob_{inicial} = \frac{\text{número_de_muestras_clase_1}}{\text{total_de_muestras}}$$

También se calcula lo que se conoce como log(odds).

$$\log(odds)_{base} = \ln\left(\frac{\text{número_de_muestras_clase_1}}{\text{número_de_muestras_clase_0}}\right)$$

No	X	Y	T	Prob. inicial	log (odds) base
1	40	52	1	0,625	0,510
2	51	69	1	0,625	0,510
3	28	14	1	0,625	0,510
4	61	72	1	0,625	0,510
5	59	34	1	0,625	0,510
6	14	85	0	0,625	0,510
7	30	73	0	0,625	0,510
8	17	69	0	0,625	0,510

$$Prob_{inicial} = \frac{5}{9} = 0,625$$

$$\log(odds)_{base} = \ln\left(\frac{5}{3}\right) = 0,510$$

Gradient Boosting

Se calculan los residuos (error cometido por el modelo).

$$\text{Residuo} = \text{Target} - \text{Predicción}$$

El residuo será utilizado para crear los siguientes modelos “débiles”

No	X	Y	T	Prob. inicial	residuo
1	40	52	1	0,625	0,375
2	51	69	1	0,625	0,375
3	28	14	1	0,625	0,375
4	61	72	1	0,625	0,375
5	59	34	1	0,625	0,375
6	14	85	0	0,625	-0,625
7	30	73	0	0,625	-0,625
8	17	69	0	0,625	-0,625

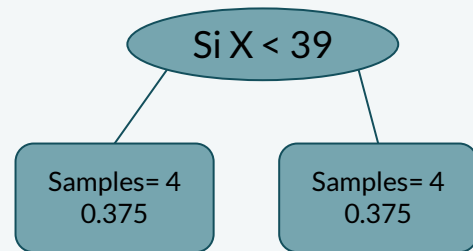
Gradient Boosting

Se genera el siguiente modelo “débil”. Este modelo es un árbol de regresión, cuyo objetivo es predecir el residuo de cada muestra.

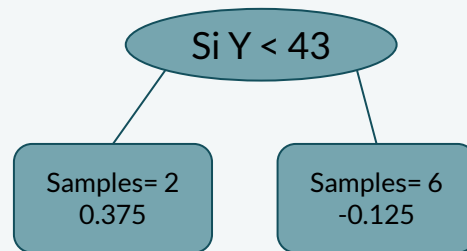
Para cada atributo del dataset, se calcula la función de pérdida (el error total que comete el modelo en todas sus predicciones). En cada nodo de decisión se elige el atributo y su valor que minimice la función de pérdida.

El Error Cuadrático Medio es uno de los más utilizados, pero hay otros.

$$ECM = \sum_{i=1}^n (Target - Predicción)^2$$



MCE=0.75



MCE=1.5



Gradient Boosting

Una vez seleccionado la condición del nodo de decisión se calcula el “valor” de cada hoja.

$$Valor = \frac{\sum_{i=1}^M Residuo_i}{\sum_{i=1}^M (Prob_{previ} a_i * (1 - Prob_{previ} a_i))}$$

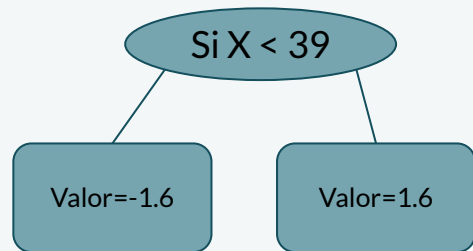
Luego se calculan las nuevas log(odds) y probabilidades de cada muestra.

$$\log(odds)_i = \log(odds)_i + Valor_i * learning_rate$$

$$Prob_i = \frac{1}{1 + \exp^{-\log(odds)_i}}$$

learning_rate es un hiperparámetro del algoritmo.

No	X	Y	T	log (odds)	Prob.
1	40	52	1	1,310	0,787
2	51	69	1	1,310	0,787
3	28	14	1	-0,289	0,428
4	61	72	1	1,310	0,787
5	59	34	1	1,310	0,787
6	14	85	0	-0,289	0,428
7	30	73	0	-0,289	0,428
8	17	69	0	-0,289	0,428



Gradient Boosting

Con las probabilidades se calculan los nuevos residuos de cada muestra.

Los nuevos residuos se utilizan para entrenar el siguiente modelo débil.

Por lo general, los árboles de decisión se los crea con un máximo de hojas entre 8 y 32, pero se puede utilizar cualquier otro criterio de parada.

No	X	Y	T	Prob.	Residuo
1	40	52	1	0,787	0,212
2	51	69	1	0,787	0,212
3	28	14	1	0,428	0,571
4	61	72	1	0,787	0,212
5	59	34	1	0,787	0,212
6	14	85	0	0,428	-0,428
7	30	73	0	0,428	-0,428
8	17	69	0	0,428	-0,428



Gradient Boosting

Con cada nuevo modelo se vuelven a calcular las nuevas probabilidades, y el algoritmo continúa hasta entrenar la cantidad de modelos prefijada.

No	X	Y	T	Prob _{inicial}	Prob ₁	Prob ₂	Prob ₃	Prob ₄	Prob ₅	Prob ₆
1	40	52	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
2	51	69	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
3	28	14	1	0,625	0,428	0,342	0,265	0,200	0,148	0,108
4	61	72	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
5	59	34	1	0,625	0,787	0,874	0,929	0,961	0,979	0,988
6	14	85	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108
7	30	73	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108
8	17	69	0	0,625	0,428	0,342	0,265	0,200	0,148	0,108

Gradient Boosting

Para realizar la predicción se utilizan los log(odds) y probabilidades que aporta cada modelo “débil”.

$$pred = \log(odds)_{base} + \sum_{i=1}^T Valor_{arbol_i} * learning_rate$$
$$Prob = \frac{1}{1 + e^{-pred}}$$

No	X	Y	pred	Prob	Respuesta
101	61	37	4,085	0,983	1
102	17	69	-0,344	0,414	0

Se redondean las probabilidades

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gbc = GradientBoostingClassifier(loss='log_loss', learning_rate=0.1,
                                max_depth=3, n_estimators=50)

model = gbc.fit(X_train, T_train)

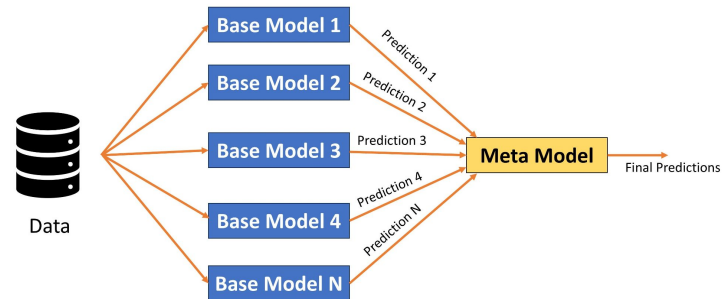
print(model.predict_proba(X_test))  # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
```

```
# Otras opciones
import lightgbm as lgb
lgbm = lgb.LGBMClassifier()
from xgboost import XGBClassifier
xgb = XGBClassifier()
```

05

Stacking



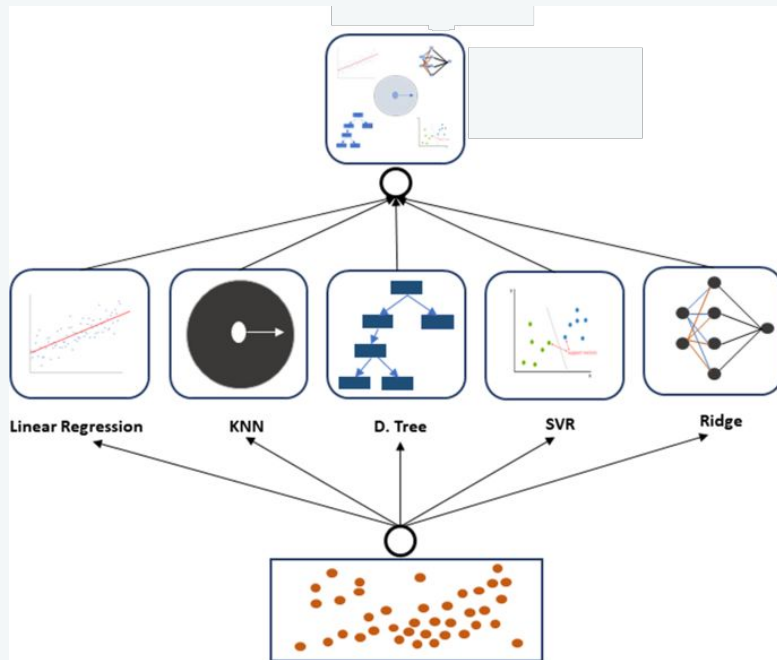
Stacking

El método de ensambles Stacking plantea el uso de múltiples estimadores de naturaleza distinta: árboles de decisión, modelos de regresión, SVM, Naïve Bayes, redes neuronales, etc.

Estos estimadores son llamados “base” o “débiles”.

Introduce el concepto de meta-aprendizaje, esto es un modelo que aprende lo aprendido por el resto de los modelos.

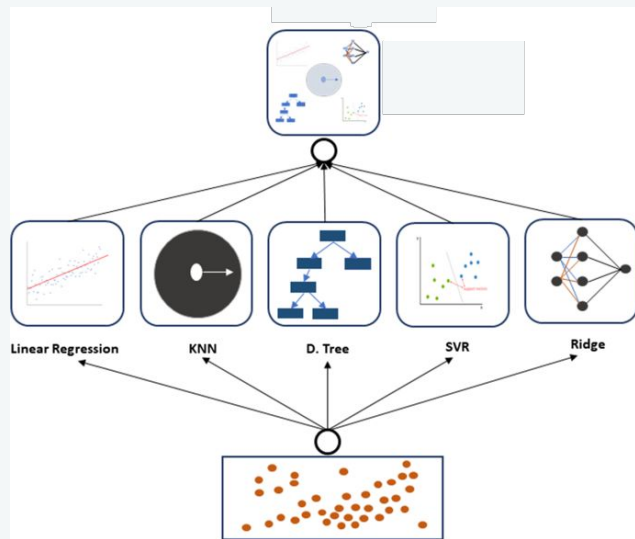
El “meta-estimador” aprende de las predicciones de los estimadores de “base”.



Stacking

Todos los estimadores aprenden de manera independiente.

Se arma un nuevo dataset, el cual está conformado con las predicciones de cada estimador.

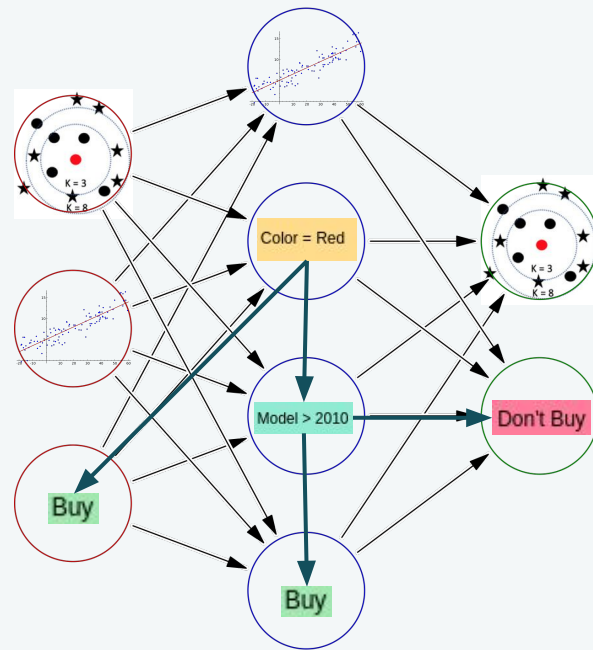


ID	X	Y	Z	T	Pred. RL	Pred. SVM	Pred. RF	Pred. RN	Pred. KNN
1	3.345	43.71	-1.73	1	0.934	0.87	0.23	0.19	0.892
2	7.21	57.2	-3.81	0	0.231	0.143	0.04	0.931	0.81
3	1.98	32.89	1.83	1	0.45	0.76	0.59	0.289	0.512
4	-0.29	85.23	2.21	0	0.001	0.128	0.014	0.003	0.109

Stacking

Como último paso se entrena el “meta-estimador” que tome como entrada las predicciones de los estimadores de “base”, para predecir la clase de cada muestra. El meta-estimador puede ser cualquiera de todos los disponibles (¡¡¡ incluyendo un ensemble !!!!)

ID	T	Pred. RL	Pred. SVM	Pred. RF	Pred. RN	Pred. KNN	Pred
1	1	0.934	0.87	0.23	0.19	0.892	0.81
2	0	0.231	0.143	0.04	0.931	0.81	0.35
3	1	0.45	0.76	0.59	0.289	0.512	0.63
4	0	0.001	0.128	0.014	0.003	0.109	0.072



Stacking

```
from sklearn.ensemble import StackingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

# Crear los estimadores base, cada uno configurado como sea necesario
base_learners = [('b11', DecisionTreeClassifier(max_depth=4)),
                  ('b12', KNeighborsClassifier(n_neighbors=5)),
                  ('b13', LogisticRegression()) ]

stck = StackingClassifier(estimators = base_learners,
                          final_estimator = RandomForestClassifier())
model = stck.fit(X_train, T_train)

print(model.predict_proba(X_test)) # devuelve la proporción de votos

# https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html
```

Stacking de múltiples capas

```
from sklearn.ensemble import StackingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression

base_learners_1 = [('b11', DecisionTreeClassifier(max_depth=4)),
                  ('b12', LogisticRegression()) ]
base_learners_2 = [('b13', KNeighborsClassifier(n_neighbors=5)),
                  ('b14', LinearRegression()) ]

stck_2 = StackingClassifier(estimators = base_learners_2,
                           final_estimator = RandomForestClassifier())
stck_1 = StackingClassifier(estimators = base_learners_1,
                           final_estimator = stck_2)
model = stck_1.fit(X_train, T_train)
```

¿Cómo está
conformado
este stacking?

Stacking

```
from sklearn.ensemble import StackingClassifier

stck = StackingClassifier(estimators = base_learners,
                          final_estimator = RandomForestClassifier(),
                          cv = 10,
                          passthrough = True)

model = stck.fit(X_train, T_train)
```

cv → Cross validation

passthrough → en True, además de las predicciones, usa las muestras originales.

Actividades

2. Utilice la base de datos “glass”, separando en 90-10.

Entrene un AdaBoost, un Gradient Boosting y un Stacking (con al menos cuatro estimadores “base”).

Calcule e imprima accuracy, precision, recall, f-score y AUC-ROC.

Repita los cálculos 10 veces, cambiando el random_state en el split. Calcule el promedio e IC_95% de cada métrica.

Compare los promedios e IC obtenidos con los de la actividad 1.

RESUMEN

- Métricas de evaluación
- Ensembles
- Bagging
- Boosting
- Stacking

